



Reporte de películas y deportes.

**Nombres y apellidos del alumno(a):
Joshua Ontiveros Cabrero**

**Nombre del docente:
Naranjo Avilez Jesus**

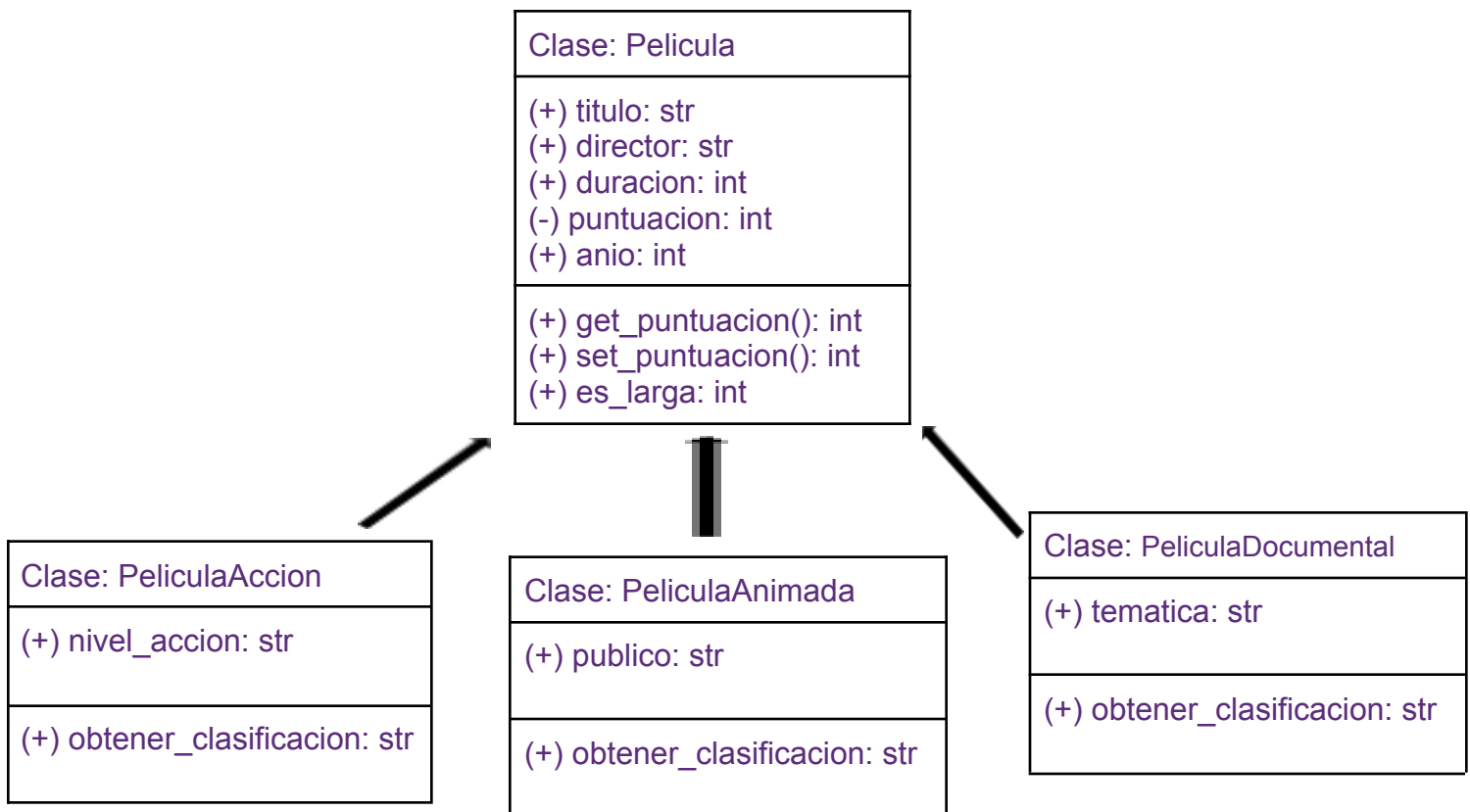
**Carrera Tecnologías de la información:
Programación orientada a objetos**

**Universidad Politécnica de Baja California.
Mexicali Baja California**

06/06/2026

Introduccion (Películas):

En el programa del día de hoy usaremos de nuevo los principios que hemos estado viendo en cada programa que hemos hecho en esta materia, los cuales son muy característicos de la programación orientada a objetos: el encapsulamiento, la abstracción, y el polimorfismo, este programa tiene una estructura muy similar a los programas hechos pero hay una pequeña diferencia, puesto que en esta ocasión la clase padre acumulo la gran mayoría de la información general de las películas y por otro lado las clases hijas solo añadieron un único atributo extra lo cual deja muy notorio el polimorfismo que fue utilizado en el programa, muchas cosas no se repitieron lo que volvió dinámico el trabajo y mejor aun, mas facil de comprender.



Preguntas de verificación

1. En el código, ¿cuál es la clase y cuáles son los objetos?
La clase es Pelicula, los objetos son las peliculas como el rey leon y titanic.
2. ¿Qué pasaría si no tuviéramos el constructor y quisiéramos ponerle título a una película?
Da error directamente pues no esta definida previamente.
3. ¿Cuántas instancias de película se crearon?
Se crearon 2 instancias.
4. ¿Qué diferencia hay entre un atributo y un método?
El atributo contiene las características que contiene el objeto mientras que el método son las acciones o funciones que puede realizar el objeto.
5. ¿Por qué el método recibe self como parámetro?
Self representa el objeto que está siendo utilizado en el momento.
6. ¿Qué pasaría si alguien intenta hacer p1.__puntuacion = 500 directamente?
El atributo original no se ve afectado conservando su valor.
7. ¿Para qué sirve la condición dentro del setter?
Sirve para validar los datos previamente guardados.
8. ¿Por qué es útil que quien usa la clase no sepa cómo está guardada la puntuación internamente?
Porque estaria aplicandose el principio de encapsulacion, protegiendo la integridad de la informacion utilizada.
9. ¿Qué atributos tiene PeliculaAccion en total, los propios y los heredados?
titulo, director, duracion, __puntuacion sumando nivel_accion.
10. ¿Para qué usamos super()?
Para llamar elementos de la clase padre
11. ¿Tendría sentido crear una clase PeliculaTerror de la misma forma?
Tendria mucho sentido puesto que comparten la misma estructura por el hecho de ser tambien una pelicula, eso si, poniendo atributos relacionados a su genero.
12. ¿Por qué ambas clases tienen el mismo nombre de método pero hacen cosas distintas?
Se aplica el principio del poliformismo, utilizan la misma funcion mas sin embargo con diferentes resultados.
13. ¿Qué ventaja tiene recorrer la lista y llamar obtener_clasificacion() sin preocuparse por el tipo de película?
El codigo se vuelve mas simple ademas de volverlo mas flexible y escalable.
14. ¿Si agregas una clase PeliculaTerror, qué tendría que devolver su método obtener_clasificacion()?
Deberia devolver una clasificacion acorde a una pelicula de terror, cada objeto deberia responder su propia clasificacion.

Primero creamos un archivo pelicula.py, esta sera nuestra clase padre, de la cual nuestras clases hijas heredaran informacion, Porque es la clase padre? Simple, la clase pelicula contiene toda la informacion general que una pelicula puede tener en comun: Su nombre, director, duracion, puntuacion y año.

pelicula.py

Aqui puede verse lo previamente mencionado, primero creamos la clase “Pelicula” y acto seguido definimos nuestro constructor, el constructor creara el self (que representa el objeto en si mismo), el titulo, el director, su duracion, puntuacion y año, Acto seguido definimos sus atributos.

```
class Pelicula:
    def __init__(self, titulo, director, duracion, puntuacion, anio):
        self.titulo = titulo
        self.director = director
        self.duracion = duracion
        self.__puntuacion = puntuacion # atributo privado
        self.anio = anio
```

Definimos la funcion “mostrar_info”, tendra la facultad de mostrarnos la informacion general de cada pelicula aplicando el principio de polimorfismo, esta funcion cambiara dependiendo la peli.

```
def mostrar_info(self):
    print("Título: ", self.titulo)
    print("Director: ", self.director)
    print("Duración: ", self.duracion, "minutos")
    print("Puntuación: ", self.__puntuacion)
    print("Año: ", self.anio)
```

A

Ahora definiremos la funcion “get_puntuacion”, get significando obtener, la funcion nos dara la informacion de que puntuacion le fue asignada a la pelicula.

```
def get_puntuacion(self):
    return self.__puntuacion
```

Ahora la funcion `set_puntuacion`, set significando “asignar”, con esta añadiremos una condicional la cual dicta “Si la puntuacion tiene un valor entre 0 a 10 entonces asigna el valor puesto en ese rango, de caso contrario muestra un error, esto debido a que debe mantener coherencia, una calificacion de 0 al 10 no debe permitir un 99 o -200 de calificacion (Aunque sabemos que algunas peliculas si merecen ese -200)”

```
def set_puntuacion(self, valor):
    if 0 <= valor <= 10:
        self.__puntuacion = valor
    else:
        print("Error: la puntuación debe estar entre 0 y 10")
```

Aqui definimos una funcion que revisa la duracion de la pelicula, si la pelicula dura mas de 2 horas (120 min) mandara un aviso al programa, un true o un false.

```
def es_larga(self):
    return self.duracion > 120
```

Acto seguido se define el “obtener_clasificacion”, esto con el fin de ver la clasificacion de la pelicula puesto que el silencio de los inocentes (por dar un ejemplo) debe tener una clasificacion C para impedir que menores la vean mientras que lilo & stitch (por dar otro ejemplo) es apto para toda la familia.

```
def obtener_clasificacion(self):
    return "AA – apta para toda la familia"
```

Ya habiendo terminado con nuestra clase padre ahora nos tocara definir las clases hijas, la primera: pelicula de accion.

 `pelicula_accion.py`

Importamos del archivo pelicula la clase Pelicula.

```
from pelicula import Pelicula
```

Definimos la clase `PeliculaAccion` que hereda de la clase `Pelicula`, definimos nuestro constructor que a parte de darnos todos los atributos generales de una pelicula tambien añadiremos un atributo propio: El nivel de accion.

```
class PeliculaAccion(Pelicula):
    def __init__(self, titulo, director, duracion, puntuacion, anio, nivel_accion):
        super().__init__(titulo, director, duracion, puntuacion, anio)
        self.nivel_accion = nivel_accion
```

Definimos la funcion que nos brinde la clasificacion de la pelicula, la pelicula sera john wick entonces la clasificacion debe ir acorde a ello, 15 años.

```
def obtener_clasificacion(self):  
    return "B15 – no recomendada para menores de 15 años"
```

Siendo que ya definimos todos los atributos de las peliculas en general, el crear las clases hijas es mucho mas rapido y dinamico, terminamos con la pelicula de accion ahora vayamos a la siguiente, creamos el archivo pelicula_animada.py

 pelicula_animada.py

Importamos del archivo pelicula la clase Pelicula.

```
from pelicula import Pelicula
```

Creamos la clase Pelicula animada, la cual heredara de Pelicula y ahora definimos nuestro constructor, hereda todo lo general de una pelicula, ahora añadimos el tipo de publico al que va dirigido. En el super enviaremos la informacion a nuestra clase Pelicula para que aplique sus procesos antes de mostrarlos en el main.

```
class PeliculaAnimada(Pelicula):  
    def __init__(self, titulo, director, duracion, puntuacion, anio, publico):  
        super().__init__(titulo, director, duracion, puntuacion, anio)  
        self.publico = publico
```

Definimos la funcion obtener_clasificacion la cual nos dara una clasificacion acorde: AA que es apto para toda la familia puesto que nuestra pelicula sera el rey leon.

```
def obtener_clasificacion(self):  
    return "AA – apta para toda la familia"
```

Estamos cerca del final, creamos el archivo pelicula_documental

 pelicula_documental.py

Importamos del archivo pelicula la clase Pelicula

```
from pelicula import Pelicula
```

Creemos la clase PeliculaDocumental que hereda de la clase Pelicula, definimos nuestro constructor que tomara informacion de la clase padre y añadiremos la tematica del documental.

```
class PeliculaDocumental(Pelicula):
    def __init__(self, titulo, director, duracion, puntuacion, anio, tematica):
        super().__init__(titulo, director, duracion, puntuacion, anio)
        self.tematica = tematica
```

Definimos obtener_clasificacion para que nos mande el tipo de clasificacion

```
def obtener_clasificacion(self):
    return "TVPG – Guia paternal sugerida"
```

Crearemos nuestro archivo main, el cual sera la goma que une todo nuestro trabajo, importaremos la clase Pelicula del archivo pelicula y haremos lo mismo con las clases hijas, importamos las clases PeliculaAccion, PeliculaAnimada, y PeliculaDocumental de los archivos pelicula_accion, pelicula_animada y pelicula_documental respectivamente.

```
from pelicula import Pelicula
from pelicula_accion import PeliculaAccion
from pelicula_animada import PeliculaAnimada
from pelicula_documental import PeliculaDocumental
```

Definimos nuestra funcion principal o “main”, este albergara la informacion que le mandaremos a nuestras clases. La p1 representa la clase pelicula, en general, es como nuestro ejemplo. pa representa PeliculaAccion, la cual sera la pelicula john wick, an representa PeliculaAnimada que sera el rey leon y finalmente PeliculaDocumental, que sera nuestro planeta.

```
def main():
    p1 = Pelicula("Titanic", "James Cameron", 194, 9, 1997)
    pa = PeliculaAccion("John Wick", "Chad Stahelski", 101, 8, 2014, "alta")
    an = PeliculaAnimada("El rey león", "Roger Allers", 88, 9, 1994, "familiar")
    doc = PeliculaDocumental("Nuestro planeta", "Alastair Fothergill", 50, 9, 2019, "naturaleza")
```

Ahora mostraremos nuestra lista que contenga las peliculas, peliculas = titanic. John wick, rey leon y nuestro planeta.

```
peliculas = [p1, pa, an, doc]
```

Aqui estariamos mostrando el formateo que deseamos que tenga nuestro programa: mostrara la informacion de la pelicula, si es larga, su clasificacion y el "-"*40 solo es para poner 40 veces el simbolo de menos que servira como un separador entre cada display.

```
for p in peliculas:
    p.mostrar_info()
    print("¿Es larga?", p.es_larga())
    print("Clasificación:", p.obtener_clasificacion())
    print("-" * 40)
```

Finalmente cerramos con `if __name__ == "__main__":` `main()` para que el programa pueda ejecutarse correctamente, asegura que solo funcione cuando lo ejecutemos directamente.

```
if __name__ == "__main__":
    main()
```

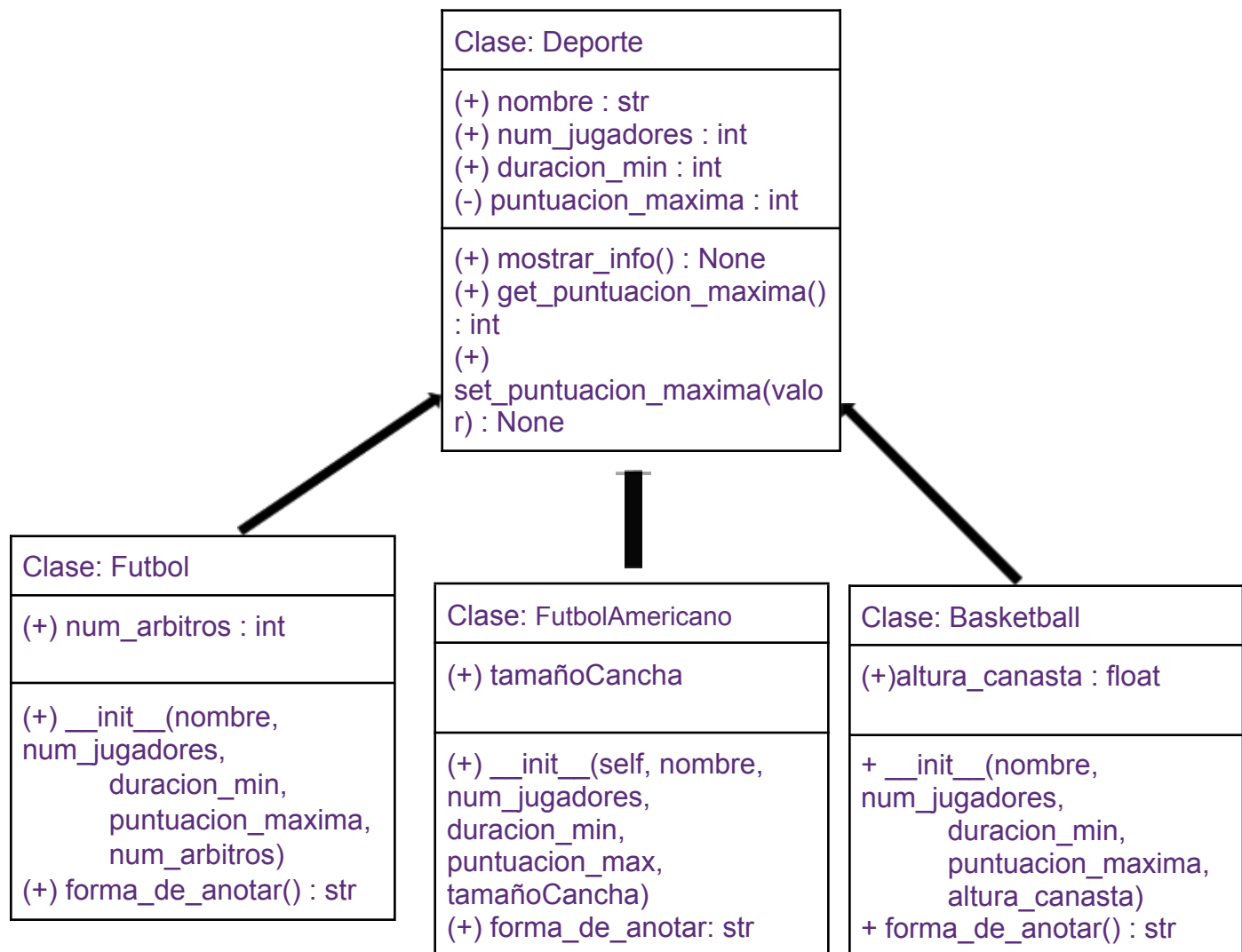
Este es nuestro producto final, el formateo quedo tal como lo mencione arriba, muestra orden, limpieza y simpleza lo que lo vuelve agradable a la vista sin perder su utilidad.

```
Título:      Titanic
Director:    James Cameron
Duración:    194 minutos
Puntuación:  9
Año:         1997
¿Es larga?   True
Clasificación: AA – apta para toda la familia
-----
Título:      John Wick
Director:    Chad Stahelski
Duración:    101 minutos
Puntuación:  8
Año:         2014
¿Es larga?   False
Clasificación: B15 – no recomendada para menores de 15 años
-----
Título:      El rey león
Director:    Roger Allers
Duración:    88 minutos
Puntuación:  9
Año:         1994
¿Es larga?   False
Clasificación: AA – apta para toda la familia
-----
Título:      Nuestro planeta
Director:    Alastair Fothergill
Duración:    50 minutos
Puntuación:  9
Año:         2019
¿Es larga?   False
Clasificación: TVPG – Guía paternal sugerida
-----
```


Introducción (Deportes):

Siguiendo los ejemplos que previamente se otorgaron para poder hacer las actividades ahora brincamos al tema de los deportes, siguiendo una estructura bastante similar a el proyecto anterior de películas se puede de igual manera, lograr un programa funcional.

Ahora usando las clases de Deporte como la clase padre, aplicamos la misma lógica que utilizamos en el proyecto de las películas, en la clase padre almacenamos toda la información general que todas las clases hijas puedan utilizar heredando de Deporte para ahorrar trabajo y darle modularidad, saltando a los deportes pues, es bastante explicativo porque todos en nuestra vida hemos tenido contacto con algun deporte y por ende tenemos una leve noción de como funcionan para plasmarlos en el programa que hicimos



Preguntas de interpretación

1. ¿Cuántas clases hay en el diagrama? Menciona sus nombres.
Hay 4 clases, deportes siendo la clase padre y las clases futbol, basketball y futbol americano son las hijas.
2. ¿Qué atributos comparten Fútbol y Básquetbol sin que aparezcan en su propia sección? ¿Por qué?
El nombre, duracion, jugadores y puntuacion maxima puesto que estan definidos ya en la clase Deporte, son cosas que ya tienen en comun entonces no necesitan ser definidas en su propia seccion.
3. ¿Cuál es el atributo privado de la clase Deporte? ¿Cómo se distingue en el diagrama?
La puntuación, pues esta marcada con un “-” que da a entender que es privada.
4. Para qué sirve el método set_puntuacion_maxima() en lugar de modificar el atributo directamente?
Para que pueda ser utilizada a la necesidad de las clases hijas, basquet, futbol soccer y americano tienen distintos limites entonces necesita ser utilizada la funcion por cada deporte independientemente.
5. El método forma_de_anotar() ¿aparece en Deporte? ¿Por qué crees que está solo en las clases hijas?
Porque cada uno tiene una forma distinta de anotar, futbol pateas la bola a una porteria, basquet lanzas la bola a una canasta y en futbol americano anotas de distintas formas, patearla a traves de los postes o llevarla al lado opuesto de la cancha.
6. Si se creara una instancia de Fútbol, ¿cuántos atributos tendría en total contando los heredados?
Serian 5 en total.

Despues de crear nuestra carpeta de archivos la cual llamamos Proyecto deportes, crearemos dentro el archivo deporte.py,



Ahora crearemos la clase Deporte

```
class Deporte:
```

Dentro de nuestra clase Deporte tenemos que definir nuestro constructor, todo aquello que se instanciara en general, les daremos un nombre, un numero de jugadores, una duracion al partido, y una puntuacion maxima que fue anotada

```
def __init__(self, nombre, num_jugadores, duracion_min, puntuacion_max):
    self.nombre = nombre
    self.num_jugadores = num_jugadores
    self.duracion_min = duracion_min
    self.__puntuacion_max = puntuacion_max # atributo privado
```

Definimos mostrar_info, esta nos brindara la informacion basica de cada deporte.

```
def mostrar_info(self):
    print("Nombre: ", self.nombre)
    print("Jugadores en el campo: ", self.num_jugadores)
    print("Duración: ", self.duracion_min, "minutos")
    print("Puntuación: ", self.__puntuacion_max)
```

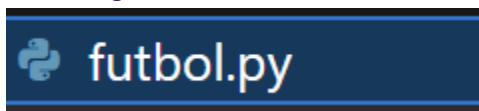
Ahora definimos get_puntuacion_max, la cual nos mandara el valor del atributo __puntuacion_max, esto porque al ser un atributo privado no puede accederse a su contenido libremente.

```
def get_puntuacion_max(self):
    return self.__puntuacion_max
```

Antes de seguir con lo siguiente hacemos una pequeña investigacion para guiar nuestra practica, el como se cuentan los puntos puesto que sera importante. Nuestros deportes anotan de distintas maneras con distintos puntajes.

```
#Anotaciones:
# (Futbol) Punto por gol: 1 punto
# (Americano) Punto por Touchdown: 6 puntos,
# 1 punto patear el balon, safety 2 puntos
# (Basquetbol) Lanzamiento desde la linea 3 puntos
# Cualquier canasta 2 puntos, tiros libres 1 punto#
```

Acto seguido crearemos nuestro archivo futbol.py.



Desde el archivo futbol importamos la informacion a la que le hicimos un formato en deporte, usaremos el from deporte import Deporte, para llevar a cabo dicha tarea.

```
from deporte import Deporte
```

Despues definimos la clase Futbol, la cual tendra herencia de Deporte. Inicializamos las mismas variables pero añadimos el atributo "arbitros", usamos super para enviar informacion.

```
class Futbol(Deporte):
    def __init__(self, nombre, num_jugadores, duracion_min, puntuacion_max, arbitros):
        super().__init__(nombre, num_jugadores, duracion_min, puntuacion_max)
        self.arbitros = arbitros
```

Definimos forma_de_annotar puesto que nuestro diagrama UML lo indicaba en sus funciones. Entonces daremos que su forma de anotar es mediante un gol.

Tambien definimos arbitros_en_partido, el cual va a mostrar cuantos hay presentes en el partido.

```
def forma_de_annotar(self):
    return "El balon entro a la porteria, se anoto un Gol"

def arbitros_en_partido(self):
    return self.arbitros
```

ÇÇ

De ahí brincaremos a basquetbol.py.

 **basquetbol.py**

Desde basquetbol.py importaremos Deporte del archivo deporte, recordemos que es nuestra clase padre; definimos nuestro constructor con los mismos datos, solo que esta vez en lugar de tener arbitros, vamos a tener la altura de la canasta.

```
from deporte import Deporte
```

```
class Basquetbol(Deporte):
    def __init__(self, nombre, num_jugadores, duracion_min, puntuacion_max, alturaDecanasta):
        super().__init__(nombre, num_jugadores, duracion_min, puntuacion_max)
        self.alturaDecanasta = alturaDecanasta
```

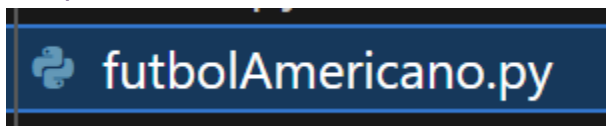
Definimos la forma de anotar tal y como indica el diagrama, añadimos condicionales para evaluar que clase de tiro se dio en base al puntaje recibido, en el if citaremos a la funcion `get_puntuacion_max()` la cual tendra el valor de la anotacion, aprovechando ese numero lo compararemos en 3 casos puesto que hay 3 puntajes que se pueden obtener si se cumple una condicion especifica.

```
def forma_de_anotar (self):
    if self.get_puntuacion_max() == 1:
        return "Se anoto un tiro libre"
    elif self.get_puntuacion_max() == 2:
        return "Se anoto un tiro normal"
    elif self.get_puntuacion_max() == 3:
        return "Se anoto un tiro desde la linea de 3 puntos"
    else:
        return "Tiro invalidado"
```

Definimos la altura de la canasta la cual es el atributo de nuestra clase basquetbol.

```
def altura_canasta(self):
    return self.alturaDecanasta
```

Ahora nos brincamos a `futbolAmericano.py`, el cual sera una ligera variacion de basquetbol.



Importamos `Deporte` (clase) de `deporte` (Archivo)

```
from deporte import Deporte
```

Ahora definimos la clase `FutbolAmericano` con herencia de `deporte` por razones obvias, definimos nuestro constructor con la misma informacion que ya hemos usado antes, pero en lugar de tener altura de canasta ahora tenemos tamaño de cancha como nuestro atributo.

```
class FutbolAmericano(Deporte):
    def __init__(self, nombre, num_jugadores, duracion_min, puntuacion_max, tamañoCancha):
        super().__init__(nombre, num_jugadores, duracion_min, puntuacion_max)

        self.tamañoCancha = tamañoCancha
```

Definimos la forma de anotar por seguir el patron que ya hemos usado, usamos condicionales citando la funcion `get_puntuacion_max()` para usar el valor como comparativa y definir el tipo de anotacion, en este caso tenemos para 1 punto, 2 puntos y 6 puntos, de caso contrario se invalida la anotacion.

```
def forma_de_anotar (self):  
    if self.get_puntuacion_max() == 1:  
        return "Se anoto un tiro PAT, la pelota atraveso los postes amarillos."  
    elif self.get_puntuacion_max() == 2:  
        return "Se anoto un Safety, el Quarterback fue tackleado."  
    elif self.get_puntuacion_max() == 6:  
        return "Se anoto un Touch Down, el jugador llego a la base enemiga."  
    else:  
        return "Tiro invalidado"
```

Definimos el tamaño de cancha y el valor que retornara sera el tamaño de la cancha en yardas.

```
def tamaño_cancha(self):  
    return self.tamañoCancha
```

Finalmente llegamos al main, el cual lo crearemos dentro de la misma carpeta “deportes”

 `main.py`

Al abrir nuestro main importaremos todo lo que hemos creado, los archivos deporte, futbol, basquetbol y futbolAmericano para utilizar sus clases, Deporte, Futbol, Basquetbol, FutbolAmericano.

```
from deporte import Deporte  
from futbol import Futbol  
from basquetbol import Basquetbol  
from futbolAmericano import FutbolAmericano
```

Definimos nuestro main, el cual tendra las variables fut, bas y ame, para resumir sus nombres y cada una contendra su informacion inherente a dichos deportes: el nombre, la cantidad de jugadores, el tiempo, la puntuacion obtenida y su atributo especial.

```
def main():
    #self, nombre, num_jugadores, duracion_min, puntuacion_max, num_arbitros
    fut = Futbol("Futbol", 22, 90, 1, 4)
    bas = Basquetbol("Basquetbol", 10, 120, 2, 3.5)
    ame = FutbolAmericano("Futbol Americano", 22, 60, 2, 100)
```

Aqui ejecutaremos todas nuestras funciones dependiendo del programa, por ejemplo, mostramos su forma de anotar y los árbitros del partido

```
fut.mostrar_info()
print(fut.forma_de_anotar())
print(f"Hay", fut.arbitros_en_partido(), "arbitros en el partido")
print("-" * 40)
```

Aqui su forma de anotar y la altura de la canasta

```
bas.mostrar_info()
print(bas.forma_de_anotar())
print(f"La canasta esta a", bas.altura_canasta(), "mts de altura")
print("-" * 40)
```

Y finalmente su forma de anotar y el tamaño de la cancha

```
ame.mostrar_info()
print(ame.forma_de_anotar())
print(f"La cancha tiene", ame.tamaño_cancha(), "yardas para ser recorridas")
print("-" * 40)
```

Finalmente cerramos con el clasico if __name__ para que nuestro programa pueda operar sin problema alguno

```
if __name__ == "__main__":
    main()
```


Este es nuestro producto final

```
Nombre:          Futbol
Jugadores en el campo: 22
Duración:        90 minutos
Puntuación:      1
El balon entro a la porteria, se anoto un Gol
Hay 4 arbitros en el partido
-----
Nombre:          Basquetbol
Jugadores en el campo: 10
Duración:        120 minutos
Puntuación:      2
Se anoto un tiro normal
La canasta esta a 3.5 mts de altura
-----
Nombre:          Futbol Americano
Jugadores en el campo: 22
Duración:        60 minutos
Puntuación:      2
Se anoto un Safety, el Quarterback fue tackleado.
La cancha tiene 100 yardas para ser recorridas
-----
```

```
Nombre:          Futbol
Jugadores en el campo: 22
Duración:        90 minutos
Puntuación:      1
El balon entro a la porteria, se anoto un Gol
Hay 4 arbitros en el partido
-----
Nombre:          Basquetbol
Jugadores en el campo: 10
Duración:        120 minutos
Puntuación:      3
Se anoto un tiro desde la linea de 3 puntos
La canasta esta a 3.5 mts de altura
-----
Nombre:          Futbol Americano
Jugadores en el campo: 22
Duración:        60 minutos
Puntuación:      6
Se anoto un Touch Down, el jugador llego a la base enemiga.
La cancha tiene 100 yardas para ser recorridas
```

Conclusión(Películas):

Para cerrar la actividad, hayq que poner una buena conclusion, me agrado bastante el programa por el hecho de que me parecio muy dinamica la manera en que se debio trabajar, un programa con el codigo hecho para que puedas resolver tus propias dudas que vayan surgiendo a lo largo del trabajo es una buena forma de aprender, se volvio bastante didactico, y por el otro lado el que se nos diera un diagrama hecho y que tuvieramos que hacer nosotros el codigo fue bastante desafiante para mi, puesto que cada quien tiene una logica propia a la hora de hacer un diagrama cosa que me generaba conflicto porque queria hacerlo a mi manera pero el trabajo indicaba que debia hacerse en base al diagrama. Siguiendo con las peliculas, hay nuevas funciones que se añadieron y me gustaron como usar un for para que nos haga un formato al mostrar la informacion del trabajo, se veia limpio y se veia ordenado o que lo volvia agradable a la vista. No tenia idea que era posible mostrar de esa forma la información, uno aprende algo nuevo cada dia.

Conclusión(Deportes):

Por el lado de deportes si se me dificulto y personalmente mi codigo no quedo como me habria gustado al 100%, puesto que no logre comprender bien la funcion para hacer la lista que muestra la informacion, deseaba añadir informacion especifica a la lista pero como cada deporte tiene un atributo propio que lo diferenciaba de los demas me generaba errores hacer dicho formato, me gusto mucho la actividad y se que tiene mucha area de mejora cosa que me emociona porque si me gustaria hacer una version mejorada usando funciones que deseaba añadir pero no cuento con las habilidades para poder implementarlas como me habria gustado.

De igual manera el programa esta hecho, esta funcional y ademas con detalles como que dependiendo del puntaje te muestra la jugada que fue realizada para obtener dicho puntaje, aun tengo mucho que aprender en este camino y con gusto estare abierto a poder aprender aun mas en esta materia tan interesante como la es la programacion orientada a objetos.